

Unidad 8

Aplicaciones a prueba: verificación y calidad del software

DESARROLLO DE INTERFACES. 2º DAM.

2.1. ¿Qué son los test?. Objetivos

- Son **programas** que ejecutan automáticamente partes del software para verificar su correcto funcionamiento.
- Utilizan frameworks de pruebas como **NUnit**, **JUnit** o **MSTest**, dependiendo del lenguaje de programación.

- Asegurar que el código funciona.
- Mejorar la calidad haciendo uso de los test diseñados.
- Control de errores más eficiente.



2.2. Definiciones

- **Caso de prueba (test case):** una unidad específica de prueba que define las **entradas**, las **condiciones iniciales**, los **pasos necesarios para realizar la prueba** y los **resultados esperados** para verificar un comportamiento o funcionalidad específica del software.
- **Prueba:** **ejecución de uno o varios casos de prueba** para evaluar si un sistema, módulo o componente cumple con los requisitos definidos, **registrando los resultados** obtenidos.
- **Error:** se produce cuando los resultados obtenidos en las pruebas no coinciden con los esperados.
- **Pruebas de caja negra:** tipo de prueba funcional, donde se ve el componente como una caja negra cuyo comportamiento sólo puede ser determinado **estudiando sus entradas y salidas**.
- **Pruebas de caja blanca:** prueba estructural donde **se analiza la lógica interna** y la estructura del código de un programa, evaluando todos los posibles caminos implementados en un programa.

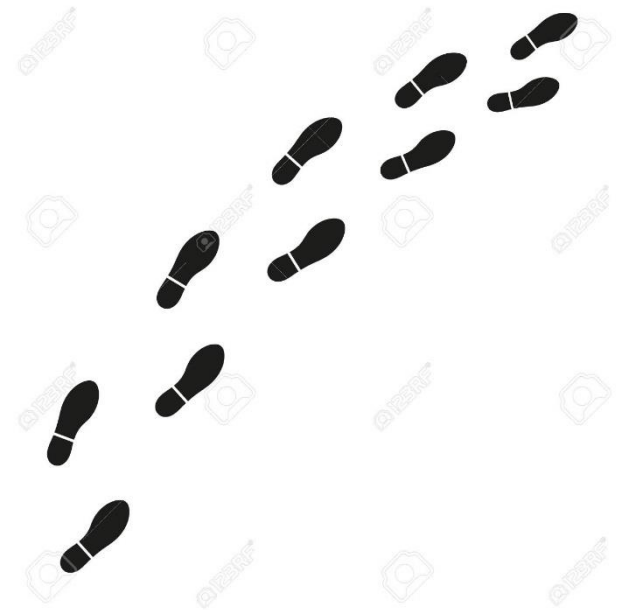
2.3. Recomendaciones



- Cada caso de prueba debe definir el resultado de salida.
- El programador debe evitar probar sus propios programas.
- Inspeccionar el resultado de cada prueba.
- Incluir datos de entradas válidos, no válidos e inesperados al generar casos de prueba.
- Evitar casos no documentados ni diseñados.
- Donde hay un defecto suele haber otros.
- Planificar las pruebas.

2.4. Tareas para probar un software

1. Diseño de las pruebas: identificar la técnica que se utilizará.
2. Generación de los casos de prueba.
3. Definición de los procedimientos de la prueba (cómo/quién/cuándo).
4. Ejecución de la prueba.
5. Realización de un informe de la prueba.



2.5. Pruebas unitarias

- ❖ Verifican el correcto funcionamiento de las unidades más pequeñas e independientes de un programa, como funciones, métodos o clases.
- ❖ Se ejecutan de forma aislada, simulando cualquier dependencia externa mediante *mocks* o *stubs*.
- ❖ Son desarrolladas y ejecutadas principalmente por los programadores durante la fase de codificación.
- ❖ "Su objetivo es identificar errores a nivel de lógica o implementación en el componente probado antes de integrarlo con otros módulos.

2.5. Pruebas unitarias

¿Qué son mocks y stubs? Diferencias?



2.6. Pruebas de integración

- Cuando los módulos de un programa funcionan correctamente es necesario probarlos conjuntamente.
- Un módulo puede tener un efecto no deseado sobre otro módulo.
- Puede ser caótico combinar todos los módulos de una vez: integración incremental.

2.6.1. Integración incremental ascendente (**bottom-up**)



- Se combinan módulos de bajo nivel en grupos que realicen una subfunción específica.
- Se escribe un controlador para coordinar la entrada y salida de los casos de prueba.
- Se prueba el grupo.
- Se eliminan los controladores y se combinan grupos del siguiente nivel de la estructura del programa.

2.6.2. Integración incremental descendente (**Top-Down**)



- Se usa el módulo del control principal como controlador de la prueba.
- Se crean módulos que simulan a los módulos que utiliza el que se está probando (resguardos=stubs).
- Se van sustituyendo los resguardos por los módulos reales (en profundidad o en anchura).
- Se llevan a cabo las pruebas cada vez que se integra un módulo.
- Tras terminar cada conjunto de pruebas, se reemplaza otro resguardo con el módulo real.

2.7. Pruebas de sistema

- Validar que el sistema completo cumple con los requisitos funcionales y no funcionales en su conjunto.
- Incluyen pruebas de interfaz de usuario, integraciones internas, rendimiento, seguridad, etc.
- Se realizan en un entorno que replica el real, pero no necesariamente el de producción.
- **Ejemplo:** probar un software de gestión asegurando que los módulos de facturación, inventario y reportes funcionan correctamente de forma conjunta.

2.8. Pruebas de regresión

- Intentan descubrir fallos resultantes por los cambios realizados.
- Tipos de regresión:
 - **Local:** los cambios introducen nuevos errores en la misma parte del sistema donde se hicieron los ajustes.
 - **Desenmascarada:** los cambios revelan errores previos.
 - **Remota:** cuando los cambios realizados en una parte del sistema afectan a otra parte aparentemente no relacionada, introduciendo errores en ella.

2.9. Pruebas de regresión

- Para mitigar los riesgos en modificaciones:
 - Repetición completa de las pruebas, de forma manual o automática.
 - Repetición parcial basada en trazabilidad y análisis de los riesgos.
 - Pruebas de cliente o usuario:
 - Beta: se distribuye a clientes potenciales y actuales.
 - Pilot: se distribuye a un subconjunto bien definido y localizado.
 - Paralela: se hace Beta y Pilot de forma simultánea.

2.10. Pruebas funcionales

- Evalúan si cada funcionalidad del software cumple con los requisitos especificados, verificando que la entrada produce la salida esperada.
- Usan los requisitos funcionales como base para crear casos de prueba.
- Prueban características visibles para el usuario (como interfaces o interacciones).
- Se deben considerar condiciones inválidas e inesperadas.
- Son una **subcategoría de las pruebas de sistema**.

Recopilando...

1. Pruebas unitarias: Se aseguran de que los componentes individuales funcionen correctamente.
2. Pruebas de integración: Verifican que los componentes interactúen de manera adecuada.
3. Pruebas funcionales: Comprueban que las funcionalidades completas cumplan los requisitos del usuario.
4. Pruebas de sistema: Evaluación global del sistema, incluyendo aspectos funcionales y no funcionales (pruebas de carga, estrés, rendimiento, etc.).
5. Pruebas E2E y de usuario (Beta/Pilot): Validan el sistema en un entorno real desde la perspectiva del usuario.
6. Pruebas de regresión: Aseguran que los cambios no hayan afectado funcionalidades previamente comprobadas.

2.11. Pruebas de capacidad y rendimiento

- Pueden demostrar que el sistema cumple los criterios de rendimiento.
- Pueden comparar dos sistemas para encontrar cuál de ellos funciona mejor.
- Pueden medir qué partes del sistema o de carga de trabajo provocan que el conjunto rinda mal.
- Es fundamental que se comiencen a realizar en el inicio del desarrollo.

2.11.1. Pruebas de capacidad y rendimiento. Tipos

- **Pruebas de carga:** validan que se alcance el objetivo de prestaciones en un entorno productivo.
- **Pruebas de capacidad:** pretenden encontrar límites de funcionamiento del sistema y detectar cuellos de botella.
- **Pruebas de estrés:** someten al sistema a una carga por encima de los límites requeridos de funcionamiento para ver cómo reacciona y cómo se recupera.
- **Pruebas de estabilidad:** comprueban que el servicio no se degrada por un uso prolongado.
- **Pruebas de aislamiento:** provocan concurrencia sobre componentes aislados para detectar errores en ellos.

2.12. Pruebas de seguridad

- Intentan validar los mecanismos de protección incorporados en el sistema.
- Datos incorrectos o inapropiados.
- Intentos de acceso no autorizados.
- Pruebas de control de la integridad de los datos.

2.13. Pruebas de usabilidad y accesibilidad

- Evalúan el producto mediante pruebas con los propios usuarios.
- Miden el nivel de cumplimiento del propósito para el que fue diseñado.
- Se selecciona un grupo de usuarios y se les pide que realicen sus tareas para las que se diseñó el programa y se anota el *feedback* proporcionado.
- No tiene por qué ser la versión final, puede ser un prototipo.
- **Las pruebas de accesibilidad intentan descubrir la facilidad con la que se puede utilizar un software.**

2.14. Prueba manual

- La prueba manual se realiza ejecutando el software, introduciendo datos de entrada e inspeccionando la salida.
- Es un proceso sencillo, no requiere aprendizaje previo y se ejecutan las pruebas como lo haría el usuario final.
- Repetitiva, susceptible de error, solo pruebas lo que ves.
- Checklist.

2.15. Prueba automática

- Se puede repetir indefinidamente.
- Si los requisitos y funcionalidades del software cambian, la prueba debe cambiar también.
- Se transmite el trabajo repetitivo al ordenador.
- Reduce los errores.
- Permite probar las partes no visibles del código.
- Uso de herramientas: Visual Studio Test, Jmeter, Bugzilla, Junit, Cucumber, Selenium, etc.

2.16. Pruebas de usuario

- Se basan en la experiencia del usuario en la realización de sus tareas.
- Cuantos más usuarios participen, se podrán detectar más situaciones de error.
- Las pruebas se ejecutan según un guion previsto.

2.17. Pruebas de aceptación

- El cliente o usuario final valida que un sistema cumpla con el funcionamiento esperado.
- Permiten que el usuario determine su aceptación desde el punto de vista de su funcionalidad y rendimiento
- Versiones alfa y beta:
 - **Prueba alfa:** la realiza un cliente en el lugar de desarrollo. El desarrollador está presente como observador, anotando errores y problemas de uso.
 - **Prueba beta:** la llevan a cabo un conjunto de usuarios en los lugares de trabajo de los clientes. El desarrollador no está presente.

2.18. Pruebas End to End (E2E)

- Validan un **flujo completo desde la perspectiva del usuario final**, abarcando todos los componentes del sistema involucrados.
- Comprueban la integración no solo dentro del sistema, sino también **entre sistemas conectados** (bases de datos, APIs externas, hardware).
- **Ejemplo:** Simular el flujo de compra en un e-commerce, desde el inicio de sesión hasta la confirmación del pago y la recepción del correo electrónico de confirmación.

Pruebas de sistema y E2E...

¿Cuáles son sus diferencias?



En conclusión



Deberían ser automatizados

Se prueban los procesos de principio a fin, con cambios en BBDD si es necesario

Se prueban procesos, pero sin modificaciones al sistema. Ej: probamos el ingreso a través de formulario de un cliente, pero sin insertarlo en la BBDD

Se testean métodos o servicios concretos de una forma individual

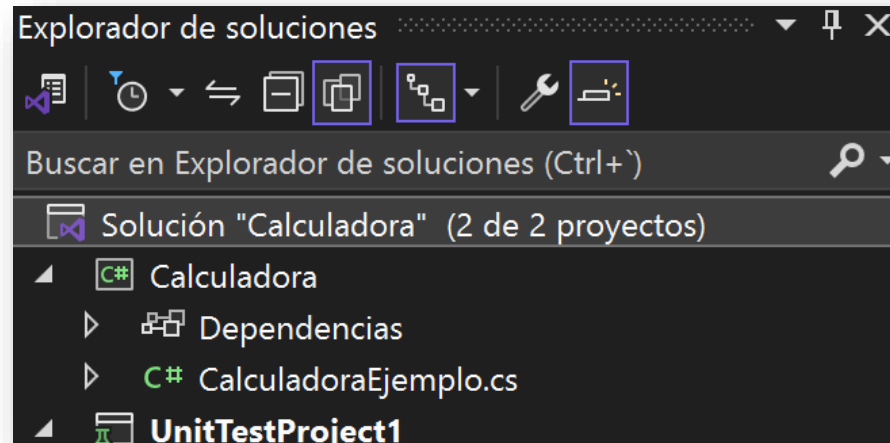
Cómo hacer un test unitario en VisualStudio

MSTest, **NUnit** y **XUnit** son frameworks de pruebas (testing frameworks) para el desarrollo de software, particularmente en el ecosistema de .NET

- **MSTest**: Es el framework de pruebas oficial de Microsoft para .NET. Está completamente integrado con Visual Studio y se utiliza principalmente para realizar pruebas unitarias y de integración.
- **NUnit**: Es un framework de pruebas de código abierto, ampliamente usado en el ecosistema .NET. Es más flexible que MSTest y ofrece una sintaxis más rica para escribir pruebas.
- **XUnit**: Este es otro framework de pruebas de código abierto, que se ha ganado mucha popularidad en los últimos años, especialmente debido a su enfoque más moderno y su mayor compatibilidad con arquitecturas asíncronas

Pruebas Unitarias: Ejemplo con MsTest

1. Crea una librería en VisualStudio con una clase estática que tenga 4 métodos: suma, resta, multiplicación y división.
2. Para ello, deberás crear un proyecto de tipo: Biblioteca de clases, que se llame Calculadora.
3. Cambia el nombre de la clase que crea por defecto a CalculadoraEjemplo:



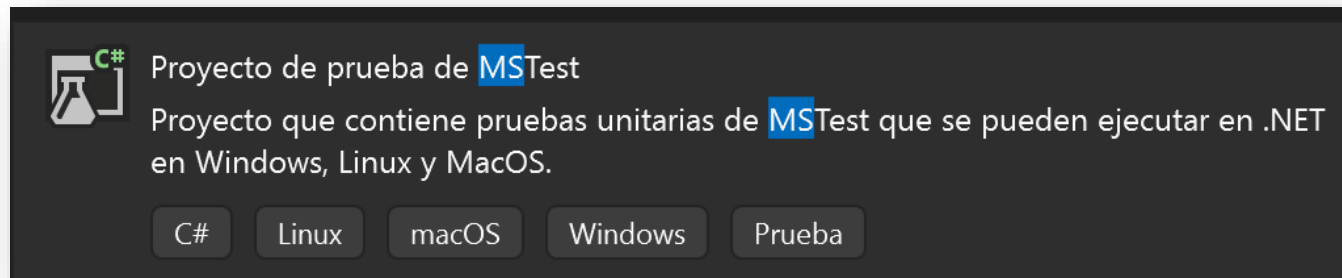
Pruebas Unitarias: Ejemplo con MsTest

La clase CalculadoraEjemplo deberá tener este aspecto:

```
namespace Calculadora
{
    2 referencias
    public static class CalculadoraEjemplo
    {
        1 referencia | ✖ 0/1 pasando
        public static int Suma (int item1, int item2)
        {
            return item1 + item2;
        }
        0 referencias
        public static int Resta (int item1, int item2)
        {
            return item1 - item2;
        }
        0 referencias
        public static int Multiplicacion (int item1, int item2)
        {
            return item1 * item2;
        }
        1 referencia | ✔ 1/1 pasando
        public static double Division (int item1, int item2)
        {
            return item1 / item2;
        }
    }
}
```

Pruebas Unitarias: Ejemplo con MsTest

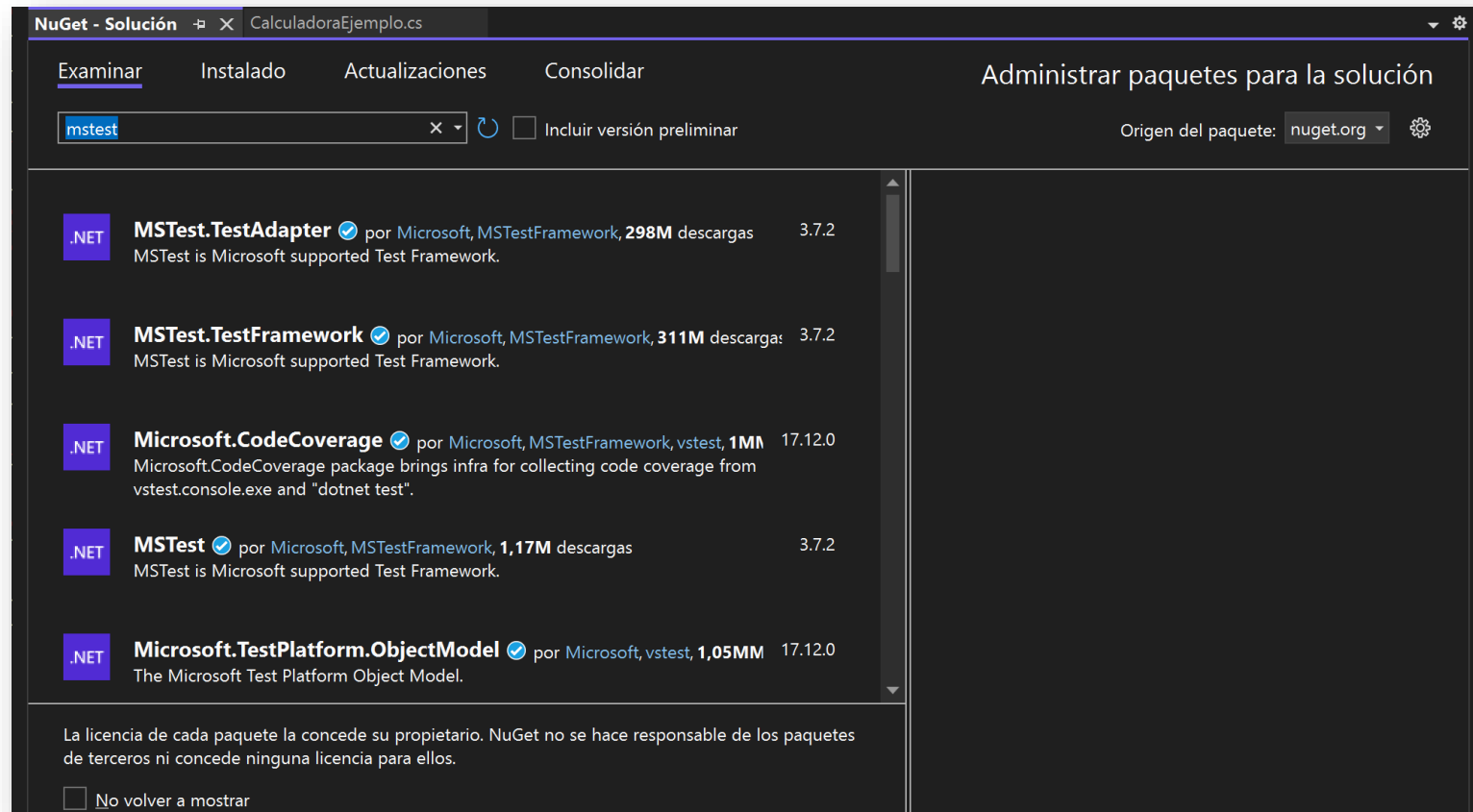
Agrega un Proyecto a la Solución de tipo:



Si no te aparece este tipo, pulsa botón derecho sobre la solución → **Administrar Paquetes NuGet para la solución**. Instala los siguientes paquetes y prueba de nuevo:

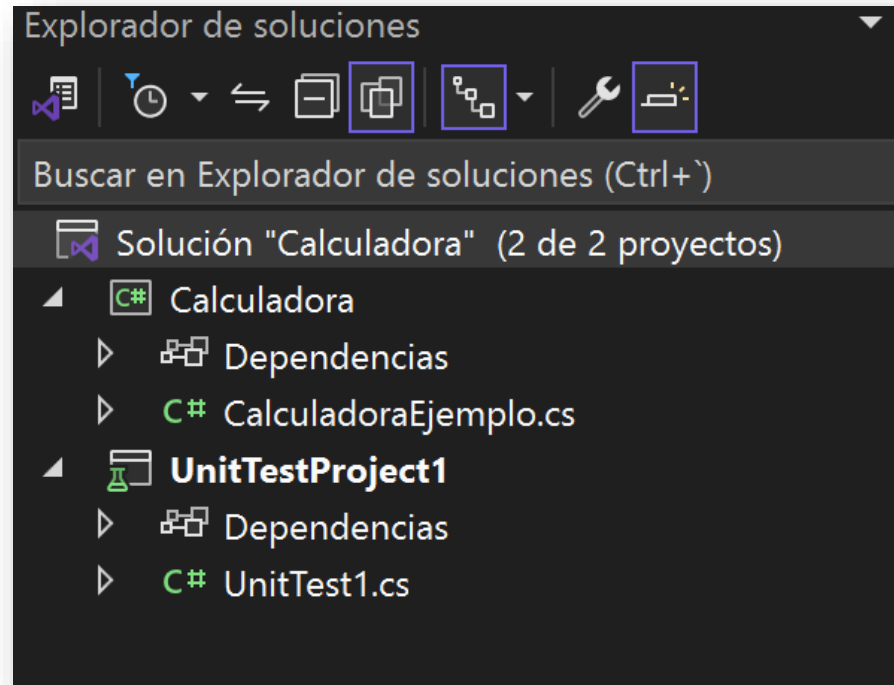
- MSTest.TestFramework
- MSTest.TestAdapter
- Microsoft.NET.Test.Sdk

Pruebas Unitarias: Ejemplo con MsTest



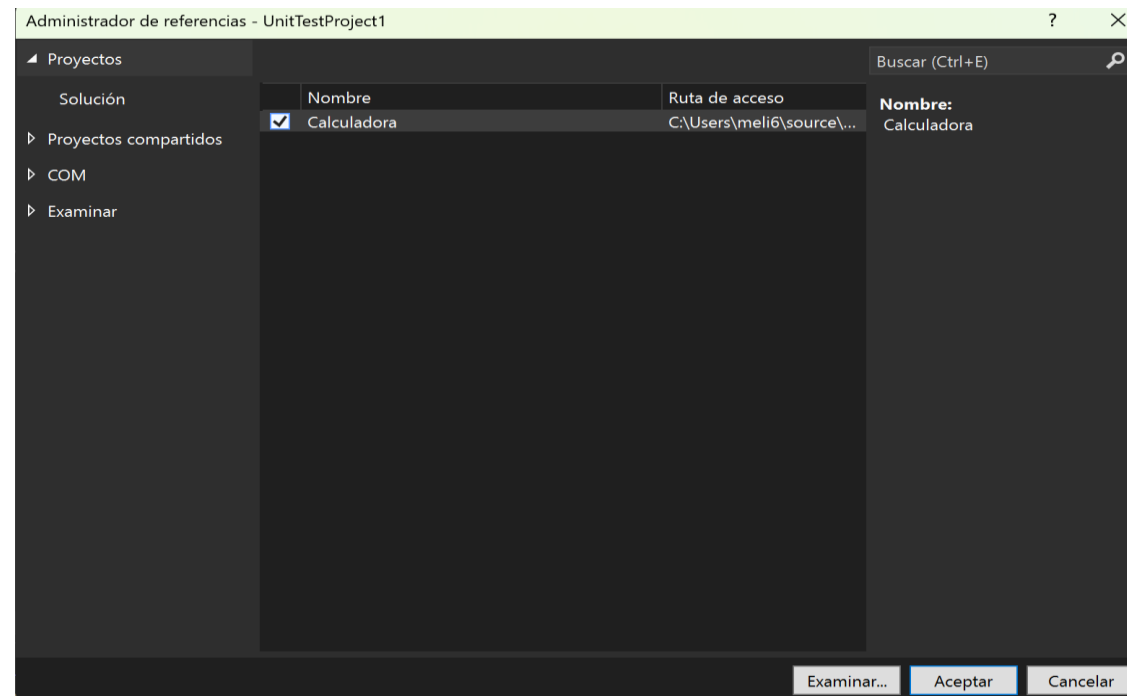
Pruebas Unitarias: Ejemplo con MsTest

Una vez hayamos agregado el proyecto, renombramos sus componentes así:



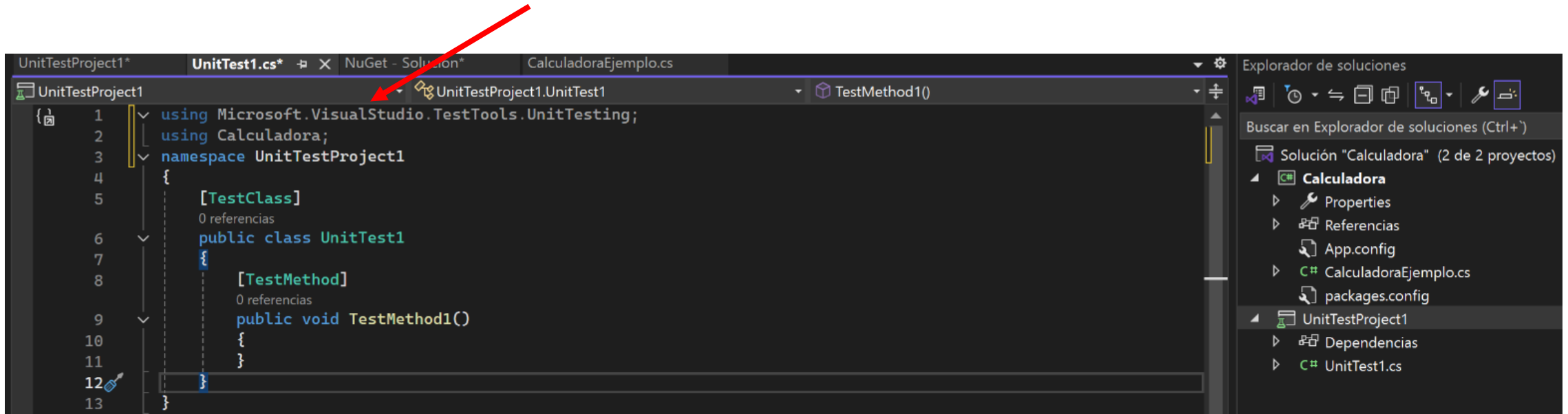
Pruebas Unitarias: Ejemplo con MsTest

Pulsamos botón derecho sobre el nuevo proyecto y añadimos como Referencia el proyecto de Calculadora, para poder usarlo en nuestra clase UnitTest1:

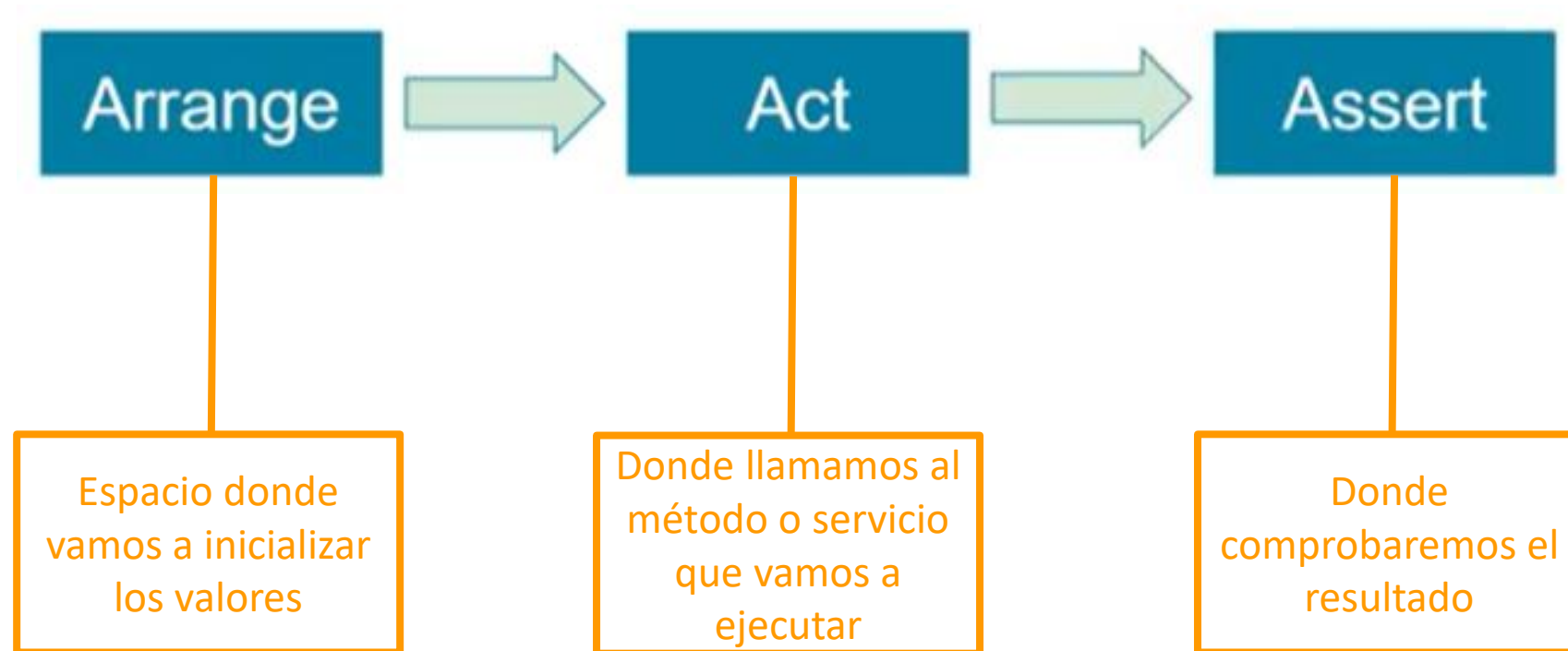


Pruebas Unitarias: Ejemplo con MsTest

Nuestra clase UnitTest1.cs tendrá este aspecto:

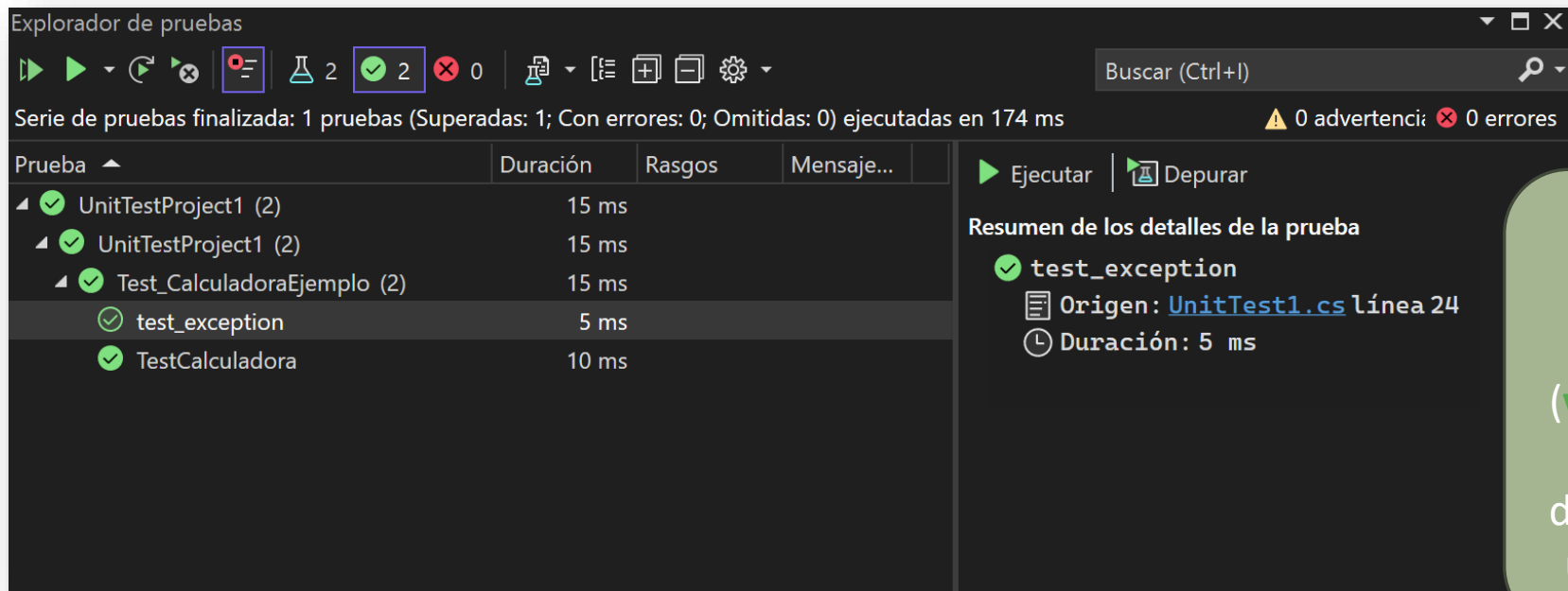


Pasos en un Test Unitario



Pruebas Unitarias: Ejemplo con MsTest

Ejecutamos las pruebas → Pulsamos botón derecho sobre [TestClass] y le damos a ejecutar pruebas:



The screenshot shows the 'Explorador de pruebas' (Test Explorer) window in Visual Studio. At the top, a status bar indicates 'Serie de pruebas finalizada: 1 pruebas (Superadas: 1; Con errores: 0; Omitidas: 0) ejecutadas en 174 ms' and '0 advertenci: 0 errores'. Below this is a table of test results:

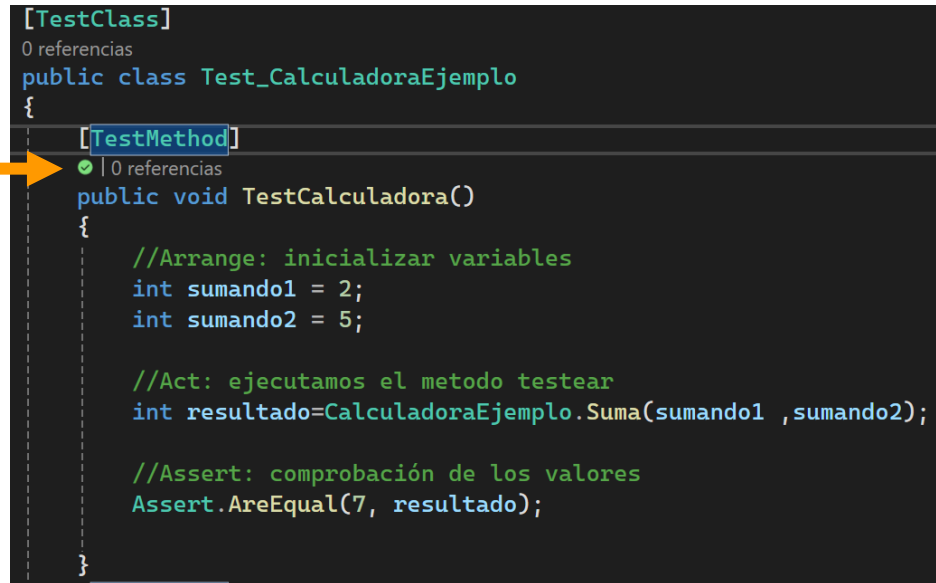
Prueba	Duración	Rasgos	Mensaje...
✔ UnitTestProject1 (2)	15 ms		
✔ UnitTestProject1 (2)	15 ms		
✔ Test_CalculadoraEjemplo (2)	15 ms		
✔ test_exception	5 ms		
✔ TestCalculadora	10 ms		

On the right, the 'Resumen de los detalles de la prueba' (Test Details) pane shows the selected test 'test_exception' with a green checkmark, its origin 'Origen: [UnitTest1.cs](#) línea 24', and its duration 'Duración: 5 ms'. The 'Ejecutar' (Run) button is highlighted.

Si el Assert se cumple, la prueba pasa (**verde**); si no, la prueba se detiene y marca un error (**rojo**)

Pruebas Unitarias: Ejemplo con MsTest

Ejecutamos las pruebas → Resultados



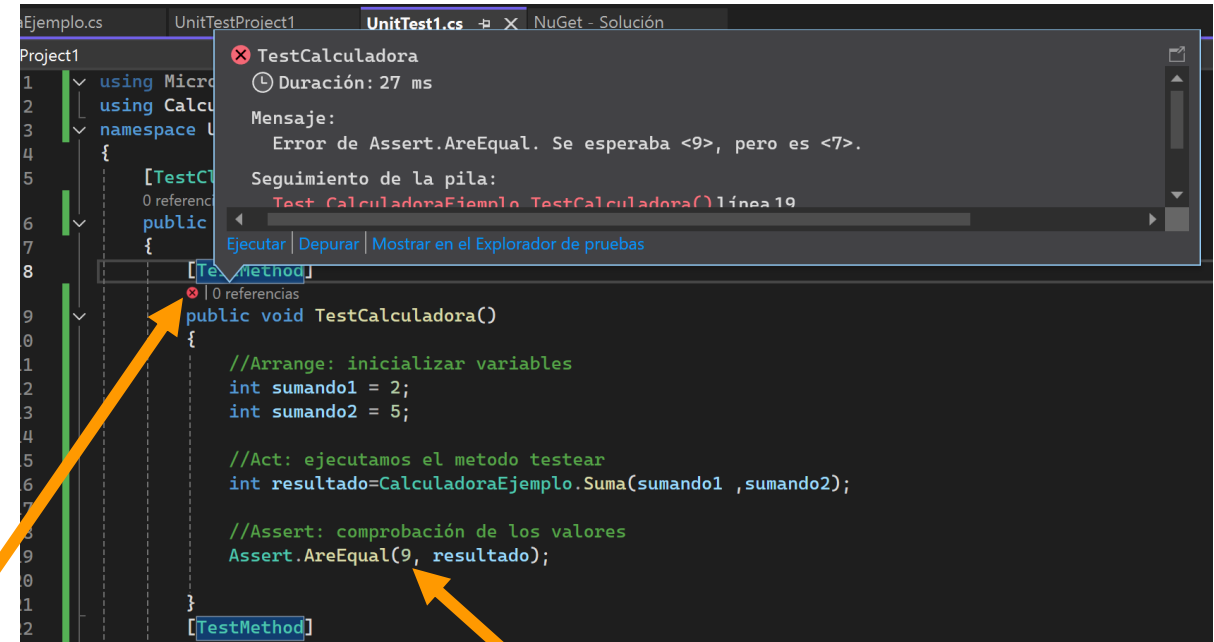
The screenshot shows a code editor with a C# test class. An orange arrow points to the green checkmark icon next to the test method name, indicating a successful execution.

```
[TestClass]
0 referencias
public class Test_CalculadoraEjemplo
{
    [TestMethod]
    0 referencias
    public void TestCalculadora()
    {
        //Arrange: inicializar variables
        int sumando1 = 2;
        int sumando2 = 5;

        //Act: ejecutamos el metodo testear
        int resultado=CalculadoraEjemplo.Suma(sumando1 ,sumando2);

        //Assert: comprobación de los valores
        Assert.AreEqual(7, resultado);
    }
}
```

Ha ido bien



The screenshot shows the same code editor as the previous one, but with a red 'X' icon next to the test method name, indicating a failure. An orange arrow points to this icon. A tooltip window is open, displaying the error details.

```
Project1
1 using Micro
2 using Calcu
3 namespace U
4 {
5     [TestC
6     0 referenci
7     public
8     {
9         [TestMethod]
10        0 referencias
11        public void TestCalculadora()
12        {
13            //Arrange: inicializar variables
14            int sumando1 = 2;
15            int sumando2 = 5;
16
17            //Act: ejecutamos el metodo testear
18            int resultado=CalculadoraEjemplo.Suma(sumando1 ,sumando2);
19
20            //Assert: comprobación de los valores
21            Assert.AreEqual(9, resultado);
22        }
23    }
24 }
```

TestCalculadora
Duración: 27 ms
Mensaje:
Error de Assert.AreEqual. Se esperaba <9>, pero es <7>.
Seguimiento de la pila:
Test_CalculadoraEjemplo.TestCalculadora() línea 19
Ejecutar | Depurar | Mostrar en el Explorador de pruebas

No ha ido bien

Tarea 1

Haz un **documento** donde registres las pruebas de todos los métodos de la librería Calculadora del ejemplo, con un caso de prueba correcto y uno incorrecto. Haz capturas de todo el proceso y explica brevemente cada paso.

El proyecto de test, a diferencia del ejemplo se llamará:
UnitTestProyect<tusIniciales> y la clase UnitTest<tusIniciales>.

Entrega este documento con tu nombre en la Tarea 1 de la UD8 de la Moodle.
Entrega también el proyecto completo de VisualStudio.

Tarea 2

Objetivo: Validar la robustez de tu aplicación mediante la creación de una batería de pruebas unitarias sobre la lógica de negocio de tu proyecto de la Unidad 3, utilizando el framework **MSTest**

Instrucciones de la Tarea

No es necesario testear toda la interfaz gráfica de la aplicación. Debes centrarte en probar la **lógica interna** (métodos que realizan cálculos, validaciones o transformaciones de datos).

Tarea 2

Instrucciones de la Tarea

1. **Selección de Lógica:** Selecciona **3 métodos** diferentes de tu proyecto (clases .cs). Estos métodos deben recibir parámetros y devolver un resultado (ej: validar un formulario, calcular un precio, formatear una cadena de texto, etc.).
2. **Configuración del Proyecto de Test:** Añade a tu solución actual un proyecto de tipo **Proyecto de prueba de MSTest** y nómbralo como `UnitTest_<TusIniciales>`. No olvides añadir la **Referencia** a tu proyecto principal para poder acceder a sus clases.
3. **Diseño de los Casos de Prueba:** Para cada uno de los 3 métodos seleccionados, debes programar **dos tests**:
 1. **Caso de éxito:** Los datos de entrada son correctos y el sistema devuelve el valor esperado (el Assert pasará a verde).
 2. **Caso de fallo/límite:** Los datos de entrada son erróneos, nulos o inesperados, y el sistema debe reaccionar según lo previsto (el Assert fallará o confirmará la detección del error).

Tarea 2

Contenido del Entregable (Documento PDF)

El documento debe estar organizado por métodos, incluyendo una breve explicación redactada de lo que sucede en cada prueba:

1. Descripción del Método.
2. Código Fuente: Captura de pantalla del método original en tu proyecto de la Unidad 3.
3. Pruebas Unitarias (Patrón AAA): Captura del código de los dos tests (éxito y fallo) donde se identifiquen claramente las secciones **Arrange, Act y Assert**.
 1. **Explicación:** Describe con tus palabras qué valores has elegido para el "Arrange" y qué resultado esperas que confirme el "Assert".
4. Evidencia de Ejecución: Captura de pantalla del **Explorador de Pruebas** tras ejecutar los tests.

El documento debe exportarse a formato **PDF**. Se debe adjuntar también el **Proyecto completo de Visual Studio** (comprimido en .zip) incluyendo tanto la aplicación como el proyecto de tests.